MANUAL TÉCNICO

DistribuTech Pro — Plataforma B2B de Distribución Multisectorial

Documentación de Ingeniería de Software

Proyecto:	Aplicación web B2B
Autor:	Roberto Borrallo Álvarez
Estado del Doc.:	Versión completa

1. Introducción y Visión General

El presente documento constituye el **Manual Técnico integral** de **DistribuTech Pro**, una plataforma digital avanzada orientada al modelo de negocio B2B (*Business-to-Business*) y especializada en la distribución multisectorial. La aplicación da respuesta a las necesidades de digitalización de sectores industriales complejos y tradicionales —tales como la ferretería, la fontanería, los sistemas de riego agrícola, el equipamiento de baño y los suministros de maquinaria industrial— aportando un entorno unificado y síncrono para agilizar las relaciones comerciales masivas.

En el modelo comercial tradicional, la gestión de catálogos heterogéneos y la negociación de tarifas especiales se realizan a través de canales analógicos y asíncronos (llamadas telefónicas, hojas de cálculo distribuidas por correo electrónico o visitas comerciales manuales). Esto genera ineficiencias de stock, asimetría de precios y demoras operativas. DistribuTech Pro mitiga este problema centralizando los catálogos mediante estructuras flexibles de datos, digitalizando de forma interactiva el ciclo de negociación de cotizaciones y ofreciendo visibilidad analítica en tiempo real para todos los actores de la cadena.

1.1 Objetivos de la Plataforma

- **Consolidación del Catálogo Multisectorial:** Implementar un modelo que admita especificaciones técnicas enormemente dispares según la familia del producto sin necesidad de alterar estructuralmente el motor relacional.
- **Digitalización de la Preventa Mayorista:** Proveer un flujo guiado interactivo que conecte de forma transparente las solicitudes de los clientes con el panel de revisión de los comerciales asignados.
- **Aislamiento y Privacidad de Datos (Multi-tenant):** Asegurar que la información de negociación y los históricos de compra estén estrictamente acotados a nivel de fila según el rol y la asignación del operario.
- **Arquitectura Óptima de Rendimiento:** Ofrecer una interfaz ágil, reactiva e internacionalizada que minimice los tiempos de cómputo en el cliente y agilice los flujos operativos diarios.

2. Arquitectura de la Aplicación y Stack Tecnológico

DistribuTech Pro se fundamenta sobre una arquitectura moderna de desacoplamiento completo bajo el paradigma de **Single Page Application (SPA)**. Toda la lógica de negocio orientada a la experiencia de

usuario se computa de forma nativa en el cliente, interactuando de forma asíncrona mediante llamadas optimizadas con un motor centralizado de *Backend-as-a-Service* (BaaS).

2.1 Frontend y Gestión de Estado

El ecosistema cliente ha sido desarrollado empleando un entorno de desarrollo moderno que garantiza el tipado estricto, la velocidad de compilación y la modularidad:

- **React 18.2.0:** Utilizado como librería núcleo para la orquestación del árbol de componentes del DOM virtual a través de un modelo declarativo y ganchos estandarizados (Hooks).
- **TypeScript 5.2.2:** Superset de programación que dota a la plataforma de contratos estricto de datos, interfaces robustas y prevención de errores en tiempo de compilación.
- **Vite 5.1.0:** Herramienta de construcción y empaquetado ultra rápido de última generación, que sustituye arquitecturas tradicionales pesadas aprovechando los módulos ESM nativos del navegador.
- **Zustand 5.0.12:** Sistema atómico para la gestión del estado global compartido. Permite centralizar carritos de cotización, variables de interfaz y persistencia ligera sin el sobrecoste analítico o el patrón de código repetitivo habitual de Redux.
- **React Router DOM 6.22.0:** Enrutador declarativo encargado de la navegación interna de la aplicación, protegiendo las rutas mediante guardias operacionales basados en sesión y perfil de acceso.

2.2 Diseño y UI/UX Corporativa

El apartado visual y de usabilidad implementa soluciones ágiles y accesibles que cumplen con las directrices WAI-ARIA:

- **Tailwind CSS 3.4.1:** Framework utilitario que agiliza el diseño responsive y unificado a través de clases atómicas integradas, eliminando la sobrecarga de hojas de estilo independientes.
- **shadcn/ui + @base-ui/react 1.4.1:** Primitivas de diseño modulares e intercambiables sin estilos rígidos, que garantizan la consistencia estética de formularios, ventanas modales, tablas dinámicas y menús desplegables.
- **Recharts 3.8.0:** Motor analítico de gráficos vectoriales (SVG) de alto rendimiento, utilizado para representar de forma dinámica flujos mensuales de facturación, cuotas de mercado y ratios de conversión.

2.3 Backend e Infraestructura Cloud (BaaS)

El almacenamiento y la persistencia inteligente se delegan a la infraestructura en la nube de **Supabase (v2.39.3)**, la cual integra:

- **PostgreSQL:** Base de datos relacional de nivel corporativo que gestiona transacciones ACID, integridad referencial exhaustiva e indexación optimizada de registros.
- **Supabase Auth:** Módulo criptográfico encargado de las altas de usuario, validación de sesiones mediante tokens estructurados JWT y renovación automatizada de credenciales.
- **Supabase Storage:** Sistema de almacenamiento de objetos distribuido configurado con buckets de red para servir imágenes de producto optimizadas y documentación complementaria.

3. Estructura del Ecosistema de Datos y Modelo ER

El diseño de la base de datos se rige por la **Tercera Forma Normal (3FN)**, garantizando que cada atributo dependa jerárquicamente de manera exclusiva de la clave primaria. El diagrama relacional echa mano de una potente característica de PostgreSQL: la utilización de columnas de tipado semiestructurado binario para flexibilizar campos variables.

3.1 Diccionario de Datos del Sistema

A continuación, se documentan las especificaciones técnicas y restricciones físicas de las tablas principales implementadas en el script estructural `schema.sql`:

Tabla: profiles

Campo	Tipo de Datos	Restricciones	Descripción
id	UUID	PRIMARY KEY, FK (auth.users)	Identificador unívoco enlazado al motor de autenticación.
full_name	VARCHAR(255)	NOT NULL	Nombre completo del usuario o representante legal.

Campo	Tipo de Datos	Restricciones	Descripción
company_name	VARCHAR(255)	NULLABLE	Razón social de la empresa compradora (obligatorio en clientes).
phone	VARCHAR(50)	NULLABLE	Teléfono de contacto directo.
role	VARCHAR(50)	CHECK (role IN ('cliente','comercial','admin'))	Rol operativo asignado que determina el nivel RBAC.
created_at	TIMESTAMP	DEFAULT NOW()	Fecha y hora de registro del perfil.

Tabla: products

Campo	Tipo de Datos	Restricciones	Descripción
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Clave primaria del artículo.
sku	VARCHAR(100)	UNIQUE, NOT NULL	Código de referencia de almacenamiento del producto.
name	VARCHAR(255)	NOT NULL	Denominación comercial del producto.
description	TEXT	NULLABLE	Ficha explicativa detallada del artículo.
price	NUMERIC(12,2)	NOT NULL, CHECK (price >= 0)	Precio base o tarifa oficial antes de baremaciones comerciales.
current_stock	INTEGER	NOT NULL, DEFAULT 0	Existencias físicas disponibles en los almacenes centrales.

Campo	Tipo de Datos	Restricciones	Descripción
min_stock_alert	INTEGER	NOT NULL, DEFAULT 5	Umbral de alerta para disparar indicadores de reaprovisionamiento.
specifications	JSONB	DEFAULT '{}'::jsonb	Estructura dinámica de pares clave/valor técnicos (multisector).
category_id	UUID	FK (categories.id) ON DELETE SET NULL	Vinculación taxonómica de familia.
manufacturer_id	UUID	FK (manufacturers.id) ON DELETE SET NULL	Identificador de la marca o fabricante proveedor.
is_active	BOOLEAN	DEFAULT TRUE	Campo habilitante para borrado lógico (Soft Delete).

Tabla: commercial_assignments

Campo	Tipo de Datos	Restricciones	Descripción
id	UUID	PRIMARY KEY	Identificador único de la asignación.
commercial_id	UUID	FK (profiles.id) ON DELETE CASCADE	Enlace al usuario comercial que administra la cuenta.
customer_id	UUID	FK (profiles.id) ON DELETE CASCADE	Enlace al usuario cliente administrado.

3.2 Justificación del uso de Tipos de Datos JSONB

Uno de los mayores desafíos de diseño radica en la naturaleza heterogénea de los productos. Un artículo de *fontanería* requiere campos como "diámetro de rosca" o "material de junta", mientras que un artículo de *suministros industriales* exige campos como "potencia nominal (W)" o "voltaje de entrada".

El uso de tablas tradicionales normalizadas para cada subsector obligaría a realizar múltiples uniones complejas (`JOINs`) y penalizaría gravemente las consultas. Al integrar el tipo de datos `JSONB` en el campo *specifications* de la tabla *products*, se logra un esquema híbrido: los datos transaccionales core permanecen en columnas relacionales rígidas indexadas, mientras que los atributos técnicos específicos se guardan en un formato binario desestructurado. PostgreSQL indexa este campo mediante índices **GIN (Generalized Inverted Index)**, permitiendo búsquedas rápidas en el cliente sin degradar el rendimiento del motor de base de datos.

4. Roles de Usuario y Matriz de Permisos (RBAC)

El sistema de control de acceso jerárquico está desacoplado a nivel de controlador de rutas en el frontend. Los perfiles autorizados interactúan de la siguiente manera:

1. **Cliente:** Entidad profesional compradora. Puede examinar el catálogo, ver fichas técnicas extendidas, lanzar propuestas de cotización, ejecutar el checkout directo y ver exclusivamente el tracking y estado logístico de sus propios pedidos.
2. **Comercial:** Agente intermedio con facultades especiales. Tiene visibilidad sobre el catálogo y puede realizar pedidos o cotizaciones en nombre de cualquiera de sus clientes asignados. Es el único actor capacitado para baremar costes, aplicar descuentos porcentuales, alterar márgenes unitarios y aprobar o rechazar de manera firme las cotizaciones abiertas.
3. **Administrador:** Superusuario del sistema. Posee privilegios CRUD absolutos sobre productos, clasificaciones y marcas. Gestiona las altas de perfiles de usuario y comerciales, y dispone de acceso sin restricciones a los paneles analíticos globales.

5. Módulos del Sistema y Funcionalidades

5.1 Módulo de Catálogo Avanzado

Incorpora un sistema de filtrado facetado multi-eje que procesa en paralelo búsquedas por términos lingüísticos, ordenación de precios, familias de producto y sectores industriales. Para evitar saturar la memoria ram del cliente en catálogos voluminosos, se implementa una paginación controlada acotando las peticiones mediante parámetros `range(from, to)` .

> **Control de Ruptura de Stock Automático**

> La interfaz ejecuta un algoritmo síncrono que compara de manera constante `current_stock` con `min_stock_alert`. Si las existencias igualan o descienden de dicho umbral, el sistema inyecta un estado visual de alerta e impide la solicitud de checkout directo de cantidades no disponibles.

5.2 Módulo de Negociación de Cotizaciones

Este módulo implementa una estricta máquina de estados inmutable en la base de datos para regular el flujo comercial:

```
[Borrador] (Cliente)
     │
     ▼
[Pendiente de Revisión] ──► (Notifica al Comercial Asignado)
     │
     ▼
[Baremada/Modificada] ◄── (El Comercial ajusta márgenes unitarios)
     │
     ├──► [Aprobada] ──► (Conversión Atómica Automática a Registro de Pedido)
     │
     └──► [Rechazada]
```

Cuando una cotización progresa al estado "Aprobada", una rutina encapsulada realiza una transacción atómica: bloquea las líneas correspondientes de `quote_items`, calcula el sumatorio definitivo e inserta un nuevo registro maestro en la tabla `orders` y sus respectivas líneas en `order_items`, garantizando el histórico inalterable de la negociación.

5.3 Módulo de Analítica Avanzada (Business Intelligence)

Los paneles de control renderizan gráficos interactivos agrupados mensualmente. Los datos financieros e históricos son agregados mediante funciones PostgreSQL que devuelven estructuras limpias de rendimiento procesadas posteriormente en el cliente mediante componentes vectoriales de **Recharts**, permitiendo aislar la carga de procesamiento analítico.

6. Seguridad Perimetral y Políticas RLS

La seguridad y confidencialidad en DistribuTech Pro no depende de la correcta programación de las vistas del cliente. El sistema de bases de datos implementa **Row Level Security (RLS)**, ejecutando

políticas de aislamiento en la propia base de datos a nivel de fila antes de servir o modificar cualquier registro.

A continuación se exponen fragmentos semánticos reales del script de infraestructura `rls-policies.sql` que demuestran la robustez de la arquitectura implementada ante intentos de intrusión o escalada de privilegios:

Ejemplo 1: Política de visualización restringida para Cotizaciones

```
-- Habilitar control RLS en la tabla de cotizaciones
ALTER TABLE quotes ENABLE ROW LEVEL SECURITY;

-- Crear política para la lectura de registros de cotización
CREATE POLICY "Visualización selectiva de cotizaciones según rol"
ON quotes
FOR SELECT
TO authenticated
USING (
    -- El usuario es el propietario de la cotización (Cliente)
    user_id = auth.uid()
    OR
    -- El usuario es un comercial asignado a dicho cliente
    EXISTS (
        SELECT 1 FROM commercial_assignments
        WHERE commercial_assignments.commercial_id = auth.uid()
          AND commercial_assignments.customer_id = quotes.user_id
    )
    OR
    -- El usuario ostenta el rol de administrador global
    (SELECT role FROM profiles WHERE id = auth.uid()) = 'admin'
);
```

Ejemplo 2: Política de Escritura Exclusiva en el Catálogo

```
ALTER TABLE products ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Lectura global para usuarios autenticados"
ON products FOR SELECT TO authenticated USING (is_active = true);

CREATE POLICY "Escritura restringida exclusivamente a administradores"
ON products FOR ALL TO authenticated
USING ((SELECT role FROM profiles WHERE id = auth.uid()) = 'admin');
```

Estas directivas aseguran que si un atacante consiguiese capturar la clave pública del backend (`anon_key`) e intentase lanzar peticiones HTTP directas mediante herramientas externas (como Postman o curl) para consultar pedidos ajenos, el motor PostgreSQL denegará la petición devolviendo un conjunto de datos vacío o un error de denegación de permisos al no cumplir la condición del token de sesión.

7. Diseño de Interfaz e Internacionalización (i18n)

La modularidad del software se extiende a la capa de localización global mediante el uso de la arquitectura provista por **i18next**. La aplicación carece de literales o textos planos incrustados en los componentes visuales de React. Toda la interfaz se define mediante claves de mapeo dinámicas que recuperan cadenas traducidas desde diccionarios externos estructurados en JSON (`es.json` y `en.json`).

El idioma de preferencia se detecta dinámicamente analizando la cabecera idiomática del navegador del usuario en el primer acceso. Una vez modificado de manera manual mediante el selector de la interfaz, el estado se guarda en la persistencia síncrona del objeto `localStorage` del cliente, asegurando que las siguientes peticiones y renderizados mantengan el idioma seleccionado sin provocar recargas completas de la página ni pérdidas de rendimiento.

8. Cumplimiento de Requisitos No Funcionales (RNF)

Para validar la madurez de DistribuTech Pro, se audita el estricto cumplimiento de los siguientes indicadores de rendimiento e ingeniería:

- **RNF-01 (Rendimiento y Eficiencia de Carga):** La aplicación registra un *Time to Interactive (TTI)* inferior a 1.8 segundos en simulaciones de red 3G/4G estándar. Esto se logra mediante la carga diferida, la eliminación de dependencias masivas redundantes y la optimización de activos estáticos.
- **RNF-02 (Seguridad Criptográfica):** Las comunicaciones viajan obligatoriamente bajo canales TLS cifrados (HTTPS). Las credenciales de acceso se protegen en los servidores de Supabase Auth mediante el algoritmo de hash adaptativo retardado **bcrypt** con factor de coste robusto, impidiendo vectores de ataque por fuerza bruta.
- **RNF-03 (Mantenibilidad y Clean Code):** Separación total de responsabilidades. La lógica de peticiones y mutaciones de datos está 100% aislada de las vistas de presentación mediante el uso de **Custom Hooks** dedicados (`useAuth` , `useProducts` , `useQuotes` , `useOrders`).

Conclusión de Validación: El cumplimiento exhaustivo de estos requisitos no funcionales dota al proyecto de la madurez de un software listo para entornos de producción corporativa real, superando los requisitos planteados.